

# Port

Karol Tomczyk, Patryk Nogaś

Czerwiec 2026

## 1 Wprowadzenie

W naszym projekcie na podstawy podejmowania decyzji, zdecydowaliśmy się zamodelować problem optymalnego cumowania floty w porcie w którym są dwa doki, które są od siebie oddalone w taki sposób, że statki największe blokują przepływ wewnątrz do portu, taki przypadek również występuje wtedy, kiedy dwa średnie statki stoją na przeciwko sobie. Naszym celem jest maksymalizacja funkcji celu, która premiuje ilość statków, które zacumowały, ale również ma na uwadze to jak duży jest ten statek. Kluczowe oczywiście w modelu było też to, że nie może statek być w dwóch miejscach naraz, ani dwa statki nie mogą być w jednym miejscu. Akurat ostatnie zdanie to truizm, za który przepraszam.

Podeszliśmy do problemu macierzowo, stworzyliśmy macierz decyzyjną. Można ją interpretować na wiele sposobów, jednym z nich jest myślenie że każdy statek ma swoją własną planszę/port  $M[s_1]...[d][c]$ , i wraz z wpływaniem kolejnych statków nakładamy, kolejne ograniczenia od  $s_1$  do  $S$ , gwarantując poprawne rozwiązanie.

Natomiast w metaheurystyce skorzystaliśmy z algorytmu genetyczny, algorytmu wprost wzorowanego na obserwacjach zjawisk w biologii. Przypomina on, jak zobowiązuje nazwa, ewolucję gatunków. Polega on na wylosowaniu dwóch rodziców (algorytmów), które potem posiadają dzieci z mutacjami jak z fallouta; Następnie mordujemy dzieci, które są porażkami życiowymi i zostawiamy te z najlepszymi rozwiązaniami (chromosomami).

Kluczowe informacje

- Decydentem jest bosman portu
- Jest to system podejmowania decyzji.
- Wejściem jest lista statków, ilość miejsc do cumowania w każdym z docków i ilość docków która zawsze wynosi 2 (dla innych argumentów nie ma sensu).

- postacią wyjścia jest rozłożenie statków w porcie, przy lekkiej gimnastyce można policzyć też funkcję celu i np. procent statków przyjętych do portów, lecz nie jest to nasz priorytet.
- Rozwiązanie przy korzystaniu z optymalizacji liniowej jest optymalne, rozwiązanie algorytmem ewolucyjnym nie musi być optymalne.

## 2 Forma matematyczna

Dane:

- Typy statków:

$typy \in 1, 2, 3$

- Liczba statków:  $S = 12$

- Liczba doków:

$D = 2$

- Liczba miejsc do cumowania w dokach:

$C = 10$

- Lista statków:  $L = [...]$

Zmienne decyzyjne:

$$M[N][D][C] \quad (1)$$

Funkcja celu:

$$\max \sum_{s=1}^S L[s] * (\sum_{d=1}^D \sum_{c=0}^C M[s][d][c]) \quad (2)$$

Ograniczenia:

$$\forall_{s \in S} (\forall_{d \in D} \forall_{c \in C} M[s][d][c] \leq 1) \quad (3)$$

W 3 zapewniamy, że statek, będzie co najwyżej raz w porcie.

$$\forall_{d \in D} \forall_{c \in C} (\forall_{s \in S} M[s][d][c] \leq 1) \quad (4)$$

W 4 zapewniamy, że nie ma dwóch statków w tym samym miejscu.

$$((\forall_{s_1 \in S} \forall_{d_1 \in D} \forall_{c_1 \in C} M[s_1][d_1][c_1] \equiv 1) \wedge (L[s_1] \equiv 3)) \implies (\forall_{s_2 > s_1} \forall_{d_2 \in D} \forall_{c_2 \leq c} M[s_2][d_2][c_2] \equiv 0) \quad (5)$$

W 5 zapewniamy, że statek o wielkości 3 blokuje część wewnętrzną portu.

$$(((\forall_{s_1 \in S} \forall_{d_1 \in D} \forall_{c_1 \in C} M[s_1][d_1][c_1] \equiv 1) \wedge (L[s_1] = 2)) \wedge (\exists_{s_2} M[s_2][\neg d_1][c_1] = 1) \wedge (L[s_2] \equiv 2))) \implies (\forall_{s_3 > s_1} \forall_{d_2 \in D} \forall_{c_3 \leq c} M[s_3][d][c_3] \equiv 0) \quad (6)$$

W 6 zapewniamy sobie, że dwa statki na przeciwko siebie również blokują wewnętrzną stronę portu.

### 3 Implementacja w Pythonie™

Implementacja w pythonie nie jest zaskakująca.

Losujemy dane do zadania w ten sposób:

```
1 dlugosci_lodek = np.random.randint(0,4,20) # 4 bo randint jest
    exclusive
2 wartosc_zalodke = {0:1, 1:6, 2:5, 3:4}
3 cena_zalodke = {0:1, 1:60, 2:70, 3:90}
```

Listing 1: Deklaracja i losowanie zmiennych

Następnie tworzymy model:

```
1 N_lodek = len(dlugosci_lodek)
2
3
4 model = gp.Model("Optymalizacja_Macierzowa")
5
6 #Funkcja kary => Jezeli wystapi ograniczenie dwoch lodek znakadamy na
    efunkcj celu ekar
7
8 M = model.addMVar(shape = (N_lodek,N_rzedow,N_slotow), vtype = GRB.
    BINARY, name= "Model portu do mapowania lodke")
9
10
11 f_cel = gp.quicksum(M[b,i,s] * (1 + wartosc_zalodke[dlugosci_lodek[b
    ]]) for b in range(N_lodek) for i in range(N_rzedow) for s in range
    (N_slotow))
12
13 model.setObjective(f_cel, GRB.MAXIMIZE)
```

Listing 2: Macierz decyzyjna

Warto zauważyć, że powyżej posiadamy funkcję kary której nie ma w modelu matematycznym, została ona dodana, aby ujednolicić implementację obu rozwiązań, gdyż dla metaheurystyki jest ona konieczna.

#### 3.1 Ograniczenia

W tym ograniczeniu 3, zapewniamy unikalność 3

```
1 #kazda lodka w jednym slotcie
2
3 for b in range(N_lodek):
4     model.addConstr(M[b,:,:].sum() <= 1)
```

Listing 3: Uniqueness

Natomiast w tym ograniczeniu 4 zapewniamy, że 4 nie ma duplikacji.

```
1 #kazdy slot przyjmuje tylko jedna lodke
2
3 for i in range(N_rzedow):
4     for s in range(N_slotow):
5         model.addConstr(M[:,i,s].sum() <= 1)
```

Listing 4: No duplication

Dzięki korzystaniu z `np.where`, udało nam się uprościć implementację modelu w kodzie.

Tutaj 5 implementujemy 5.

```
1 for boat_3 in np.where(dlugosci_lodek == 3)[0]:
2     for i in range(N_rzedow):
3
4         model.addConstr(
5             (M[boat_3,i,0] == 1) >> (M[:,i,1] == 0),
6             name = 'Zakaz stawania na przeciwko (dla lewej
7             strony)'
8         )
9
10        model.addConstr(
11            (M[boat_3,i,1] == 1) >> (M[:,i,0] == 0),
12            name = 'Zakaz stawania na przeciwko (dla prawej
13            strony)'
14        )
```

Listing 5: Statek wielkości 3 blokuje część portu

Natomiast tutaj 6 implementujemy 6

```
1 boat_2 = np.where(dlugosci_lodek == 2)[0]
2
3 for i in range(N_rzedow - 1):
4     for b1 in boat_2:
5         for b2 in boat_2:
6             if b1 != b2:
7
8                 z = model.addVar(vtype=GRB.BINARY, name='Wartosc
9                 blokady przez dwie lodki o dlugosci 2')
10
11                 model.addConstr(z == gp.and_(M[b1,i,0], M[b2,i,1]))
12
13                 model.addConstr((z==1) >> (M[:,i+1,:].sum() == 0))
```

Listing 6: Statek wielkości 2 blokuje część portu

I tak o to wyglądają kluczowe elementy implementacji w pythonie.

## 3.2 Wyniki

Seed dla wszystkich wyników w tej podsekcji wynosi 67.

```
1 [[1. 1.]
2  [3. 0.]
3  [2. 1.]
4  [2. 1.]
5  [0. 3.]
6  [2. 1.]
7  [1. 0.]
8  [3. 0.]
9  [0. 3.]
10 [2. 1.]]
```

Listing 7: Wyniki dla  $N\_rzedow = 10$ ,  $N\_slotow = 2$  i 20 łódek

Funkcja celu wynosi 26 dla powyższego przypadku

```
1 [[1. 2.]
2  [3. 0.]
3  [3. 0.]
4  [1. 1.]
5  [2. 1.]
6  [1. 2.]
7  [1. 1.]
8  [0. 3.]
9  [3. 0.]
10 [0. 3.]]
```

Listing 8: Wyniki dla  $N\_rzedow = 10$ ,  $N\_slotow = 2$  i 18 łódek

Funkcja celu wynosi 28 dla powyższego przypadku.

```
1 [[1. 1.]
2  [3. 0.]
3  [0. 1.]
4  [3. 0.]
5  [1. 2.]
6  [3. 0.]
7  [0. 3.]
8  [1. 1.]
9  [0. 3.]
10 [1. 2.]]
```

Listing 9: Wyniki dla  $N\_rzedow = 10$ ,  $N\_slotow = 2$  i 16 łódek

Funkcja celu wynosi 26 dla powyższego przypadku.

Widzimy, że wraz ze spadkiem ilości wszystkich łódek jest mniej łódek o długości 2 – które są najbardziej optymalną kombinacją w naszych algorytmach, ale wcale nie widzimy straty w funkcji celu. Zwiększa się natomiast ilość łódek o długości 3.

### 3.2.1 Ciekawe przypadki

```
1 [[2. 1.]
2  [1. 1.]
3  [1. 2.]
4  [1. 1.]
5  [1. 1.]
6  [1. 1.]
7  [1. 2.]
8  [1. 1.]
9  [1. 1.]
10 [1. 2.]]
```

Listing 10: Wyniki dla  $N\_rzedow = 10$ ,  $N\_slotow = 2$  i 67 łódek

Funkcja celu wynosi 24 dla powyższego przypadku. Nie wzrosła ona od przypadku z 22 i 20 łódkami, bo posiadamy funkcję kary. Nie warto zastanawiać się długo nad tym przypadkiem, bo raczej mało możliwe jest to że przyplynie ilość statków, która jest 3 krotnie większa niż jest miejsc w porcie.

## 4 Algorytm Genetyczny

Algorytm Genetyczny to metaheurystyka używana w problemach optymalizacji i podejmowania decyzji. Metoda ta wzorowana jest na biologicznej ewolucji, krzyżowaniu i mutowaniu genotypów. Genotyp składa się z chromosomów, który składa się z genów. W naszym modelu podejmowania decyzji za chromosom przyjęliśmy port, a należący do niego gen to łódka o znanej długości i pozycji w porcie (rząd i strona). Działanie algorytmu, z uwzględnieniem założeń projektowych, wygląda następująco:

1. Losowana jest populacja. W tym przypadku będą to chromosomy, które odzwierciedlają przykładowe pozycje ułożenia statków (genów) w porcie.
2. Populacja poddawana jest ocenie za pomocą pomocniczej funkcji fitness, której działanie odzwierciedla funkcję celu (blokady portu są karane, a dobre ustawienie premiowane). Najlepiej dopasowane chromosomy biorą udział w dalszym procesie ewolucyjnym
3. Genotypy wybranych chromosomów poddawane są:
  - **Krzyżowaniu:** Dwa chromosomy dzielone są na 3 równe części. Wyciągany jest środek z jednego chromosomu i zamieniany jest ze środkiem drugiego.
  - **Mutacji:** Losowo wybrany gen zostaje zmieniony na inny. Przyporządkowywana jest mu inna pozycja w porcie. Ilość elementów poddawanych mutacji zależy od podanego prawdopodobieństwa zmian.
4. Rodzi się następne pokolenie. Populacja jest aktualizowana o chromosomy, których funkcja celu jest największa. Chromosomy o mniejszej wartości odpadają. Jeżeli nie znaleziono dobrego rozwiązania, algorytm powraca do kroku drugiego (ocena nowej populacji). W przeciwnym wypadku wybieramy najlepszego osobnika (chromosm) z populacji. Jego genotyp to nasze optymalne metaheurystyczne rozwiązanie.

### 4.1 Wyniki

Implementacja zaczyna się od stworzenia populacji początkowej. Odpowiedzialna jest za to funkcja generate chromosome (patrz Listing 12). W innym modelu generowana jest lista łodzi, które wiemy, że przybędą do portu.

```
1
2 boat = [0,1,2,3]
3 wagi = [0.10, 0.35, 0.35, 0.20]
4
5 wylosowany = random.choices(boat, weights=wagi, k=N_lodek)
6
```

```

7 print(wylosowany)
8 # [1, 1, 2, 2, 1, 1, 1, 3, 3, 3, 3, 3, 1, 1, 1, 3, 1, 3, 2, 2, 2]
9
10 stala_lista = [1, 1, 2, 2, 1, 1, 1, 3, 3, 3, 3, 3, 1, 1, 1, 3, 1, 3, 2, 2, 2, 2]

```

Listing 11: Moduł odpowiadający za generowanie listy łódek

Wartość stala-lista (patrz Listing 11) importowana jest do modułu metaheurystyki. Na jej podstawie funkcja generująca chromosomy będzie tworzyła populację początkową. W celu ochrony przed tworzeniem się powtarzających chromosomów, lista jest tasowana na samym wejściu do funkcji.

```

1
2 def generate_chromosome(idx_boat, N_lodek = 21, N_rzedow = N_rzedow,
   N_slotow=N_slotow):
3
4     gene = []
5     random.shuffle(idx_boat)
6
7     zajete_sloty = set()
8     zajete_przez_trojki = set()
9
10    for b in idx_boat:
11
12        wolne = []
13        for i in range(N_rzedow):
14            for j in range(N_slotow):
15                if (i,j) in zajete_sloty:
16                    continue
17                if i in zajete_przez_trojki:
18                    continue
19                if b == 3:
20                    if (i, 1 - j) in zajete_sloty:
21                        continue
22                wolne.append((i,j))
23        if not wolne:
24            continue
25
26        rzad, strona = random.choice(wolne)
27        gene.append(Gen(b, rzad, strona))
28        zajete_sloty.add((rzad, strona))
29
30        if b == 3:
31            zajete_przez_trojki.add(rzad)
32
33    return Chromosom(gene)

```

Listing 12: Funkcja odpowiadająca za generowanie chromosomów

Przed wykonaniem kroków algorytmu trzeba było stworzyć funkcję odpowiedzialną za krzyżowanie, mutacje oraz obliczanie wartości fitness. Zaczynamy od funkcji crossing, która przyjmuje dwa chromosomy rodziców, które wyprodukują dziecko. Dziecko to wynik krzyżowania oraz również nowy członek naszej populacji. Chromosomy dzielone są na 3 równe części.

Zwracany jest nowy gen, który składa się z dwóch części chromosomu jeden oraz jednej części chromosomu 2.

```
1
2 def crossing(parent_a: Chromosom, parent_b: Chromosom):
3     '''
4     Crossing dzieli chromosomy na 3 czesci. Nastepnie zamienia ąsrodkow
5     czesc miedzy genami
6     '''
7     geny_a = parent_a.geny
8     geny_b = parent_b.geny
9     m = min(len(geny_a), len(geny_b))
10
11     p1, p2 = sorted(random.sample(range(m), 2))
12
13     segment = geny_a[p1:p2]
14     zajete = {(g.rzad, g.strona) for g in segment}
15
16     dopelnienie = [
17         Gen(g.lodka, g.rzad, g.strona)
18         for g in geny_b
19         if (g.rzad, g.strona) not in zajete
20     ]
21
22     nowe_geny = dopelnienie[:p1] + segment + dopelnienie[p1:]
23
24     return Chromosom(nowe_geny)
```

Listing 13: Funkcja odpowiadająca za krzyżowanie się chromosomów

Funkcja mutacji w sposób losowy zamienia miejscami dwa geny w chromosomie. Prawdopodobieństwo tych zmian jest odgórnie ustalone przez wartość prob, która bazowo nosi wartość 0.1. Zwracany jest chromosom z nowym układem genów.

```
1
2 def mutation(chromosome: Chromosom, N_rzedow: int, prob=0.1):
3     '''
4     Wybieramy losowy gen z chromosomu i wybieramy dla niego nowe losowe
5     miejsce
6     '''
7     geny = [Gen(g.lodka, g.rzad, g.strona) for g in chromosome.geny]
8
9     for idx in range(len(geny)):
10         if random.random() > prob:
11             continue
12
13         drugi_idx = random.randint(0, len(geny) - 1)
14         if drugi_idx == idx:
15             continue
16
17         geny[idx].rzad, geny[drugi_idx].rzad = geny[drugi_idx].rzad,
18         geny[idx].strona, geny[drugi_idx].strona = geny[drugi_idx].
19         strona, geny[idx].strona
```



```
19     return Chromosom(geny)
```

Listing 14: Funkcja odpowiedzialna za mutacje chromosomów

Funkcja fitness to pomocnicza funkcja, która działa podobnie jak funkcja celu. Oblicza ona wynik krzyżowania i mutacji dla przyjętych kryteriów. Według naszego modelu, funkcja karać będzie wszystkie chromosomy, w których występuje blokada reszty genów (dalszej części portu) tak jak opisano we wprowadzeniu. Zwracana jest wartość funkcji fitness, którą każdy chromosom ma zapisany w sobie (atrybut obiektu).

```
1
2 def fitness_func(chromosome: Chromosom):
3
4     kara_blokada = 5
5     kara_kolizacja = 10
6
7     kara = 0
8
9     pozycje = [(gen.rzad, gen.strona) for gen in chromosome.geny]
10
11     for pos in pozycje:
12         if pozycje.count(pos) > 1:
13             kara += kara_kolizacja
14
15     blokady = block_dock(chromosome)
16
17     for gen in chromosome.geny:
18         kara += sum(blokady[j] for j in range(gen.rzad)) * kara_blokada
19
20
21     wartosc = sum(wartosc_zalodka[gen.lodka] for gen in chromosome.
22                  geny)
23
24     chromosome.fitness += wartosc - kara
25
26     return chromosome.fitness
```

Listing 15: Funkcja odpowiedzialna za kontrolowanie jakości tworzonych członków populacji

Funkcja pomoczenia block-dock informuje funkcję fitness o występowaniu blokad, dzięki czemu może ona nałożyć na rozwiązanie karę.

```
1
2 def block_dock(chromosome):
3
4     occ = [[0,0] for _ in range(N_rzedow)]
5
6     for gen in chromosome.geny:
7         rozmiar = gen.lodka
8         occ[gen.rzad][gen.strona] = rozmiar
9
10    blokady = []
11    for j in range(N_rzedow):
12        suma = occ[j][0] + occ[j][1]
```

```

13     trojka_sama = (occ[j][0] == 3 and occ[j][1] == 0) or (occ[j][1]
14 == 3 and occ[j][0] == 0)
15     blokada_twarda = suma >= 4 and not trojka_sama
16
17     blokady.append(1 if blokada_twarda else 0)
18
19     return blokady

```

Listing 16: Funkcja odpowiedzialna za wybór najlepszych członków populacji

Funkcja wybierania decyduje, którzy kandydaci są najbardziej wartościowi dla naszego modelu patrząc na ich wartości fitness. Chromosomy z największą wartością fitness są najlepszymi chromosomami w populacji.

```

1 def selection(populacja: list[Chromosom], k = 3) -> Chromosom:
2
3     candidates = random.sample(populacja, k)
4     return max(candidates, key = lambda ch: ch.fitness)

```

Listing 17: Funkcja odpowiedzialna za wybór najlepszych członków populacji

Główna funkcja algorytmu genetycznego wykonuje wszystkie należyte kroki potrzebne do znalezienia doskonałego chromosomu. Zaczynamy od zapisania populacji początkowej. Zainicjowania podstawowej wartości funkcji fitness oraz określenie najlepszego chromosomu. Następnie dla N ustalonych etapów sprawdzamy populację w poszukiwaniu najlepszego chromosomu. Znaleziony chromosom jest porównywany z zapisanym chromosomem. Jeżeli znalezione rozwiązanie jest lepsze, dodajemy 1 do warunku stopu, w innym przypadku nie robimy nic. Następnie inicjujemy nową populację. Robimy to za pomocą selekcji najlepszych osobników, krzyżowania i mutacji.

```

1 def genetic_algorithm(population: list[Chromosom], etapy = 100,
2     warunek_stopu = 20):
3
4     populacja = copy.deepcopy(population)
5
6     best_fit_func = float('-inf')
7     best_chromosome = None
8     no_changes = 0
9
10    for gen in range(etapy):
11
12        for chrom in populacja:
13            chrom.fitness = 0
14            chrom.fitness = fitness_func(chrom)
15
16        print(f'Fitness w nowej populacji: {[ch.fitness for ch in
17            populacja]}')
18
19        #najlepsze chromosomy
20
21        best_in_gene = max(populacja, key=lambda ch: ch.fitness)
22
23        print(f'max_idx = {best_in_gene}')

```

```

24     if best_in_gene.fitness > best_fit_func:
25         best_fit_func = best_in_gene.fitness
26         best_chromosome = copy.deepcopy(best_in_gene)
27         no_changes = 0
28     else:
29         no_changes += 1
30
31     print(f'Generacja {gen:3d} | fitness: {best_fit_func:.2f} | bez
32     poprawy: {no_changes}')
33     print(f'Najlepszy chromosom: {best_chromosome}')
34
35     if no_changes >= warunek_stopu:
36         print(f'Zbieno w generacji {gen}')
37         break
38
39     # tworzenie nowej populacji
40     new_population = [copy.deepcopy(best_chromosome)]
41
42     while len(new_population) < len(populacja):
43
44         # selekcja turniejowa rodziców
45         parent_a = selection(populacja)
46         parent_b = selection(populacja)
47
48         # krzyzowanie
49         child = crossing(parent_a, parent_b)
50
51         # mutacja
52         child = mutation(child, N_rzedow)
53
54         new_population.append(child)
55
56     populacja = new_population
57
58     print(f'Dlugosci chromosomów: {[len(ch.geny) for ch in
59     populacja]}')
60
61     best_chromosome_mapped = chromosome_to_matrix(best_chromosome)
62
63     max_value = f_cel_value(best_chromosome)
64     print(f'Wartosc funkcji celu wynosi: {max_value}')
65
66     return best_chromosome_mapped

```

Listing 18: Funkcja generująca wyniki dla algorytmu genetycznego

Wyniki dla metaheurystyki prezentują się następująco.

```

1
2 #wartosc_za_lodke = {0:1, 1:5, 2:5, 3:5}źść
3
4 Zbieno w generacji 20ś
5 Wartoc funkcji celu wynosi: 28
6 [[1 2]
7  [0 3]
8  [2 1]
9  [1 2]
10 [1 2]

```

```

11 [0 3]
12 [2 1]
13 [1 1]
14 [1 1]
15 [3 0]]

```

Listing 19: Wynik 1 dla metaheurystyki

```

1
2 #wartosc_za_lodke = {0:1, 1:5, 2:5, 3:5}żść
3
4 Zbieno w generacji 39ś
5 Wartoc funkcji celu wynosi: 17
6 [[0 1]
7 [1 1]
8 [1 1]
9 [1 1]
10 [1 1]
11 [1 1]
12 [1 0]
13 [1 1]
14 [1 0]
15 [1 1]]

```

Listing 20: Wynik 2 dla metaheurystyki

Jak widać, wynik rozwiązania metaheurystycznego jest zależny od wagi, którą przypiszemy pojedynczym łódkom oraz rodzaju wylosowanych łódek. Jesteśmy świadomi, że da się stworzyć lepszy system obsługi łódek, rozbudowując i modyfikując funkcję celu. W prezentowanym zadaniu skupiliśmy się jednak, o wiele bardziej, na poprawnej implementacji algorytmu genetycznego w oparciu o ustalone we wstępie założenia.

## 5 Porównanie dla tych samych argumentów

```

1
2 #wartosc_za_lodke = {0:1, 1:5, 2:5, 3:5}
3
4 ## Alg. Genetyczny
5 Zbieznosc w generacji 20
6 Wartosc funkcji celu wynosi: 28
7 [[1 2]
8 [0 3]
9 [2 1]
10 [1 2]
11 [1 2]
12 [0 3]
13 [2 1]
14 [1 1]
15 [1 1]
16 [3 0]]
17
18 ## Gurobi
19 Wartosc funkcji celu wynosi: 27

```

```

20 [[1. 0.]
21 [3. 0.]
22 [2. 1.]
23 [2. 1.]
24 [0. 3.]
25 [2. 1.]
26 [1. 1.]
27 [3. 0.]
28 [0. 3.]
29 [2. 1.]]

```

Listing 21: Przyjete argumenty i wyniki

## 6 Podsumowanie

Rozwiązując problem portowy, który jest problemem kombinatorycznym i decyzyjnym, skorzystaliśmy z dwóch technik – optymalizacja liniowa (dzięki gurobi<sup>TM</sup>) oraz algorytm ewolucyjny (metaherustyka). Wyniki obu rozwiązań są zaskakująco podobne.

Rozwiązanie 1 jest ciężko skalowalne, z każdym kolejnym statkiem rośnie znacznie wykorzystanie zasobów takich jak ram i CPU, solvery takie jak gurobipy<sup>TM</sup> generują też więcej niż tylko jedno rozwiązanie co dodatkowo zwiększa koszt.

Natomiast rozwiązanie przy algorytmu genetycznego jest prościej skalowalne i tańsze (sprzętowo) w implementacji.

## 7 Disclaimers

Python<sup>TM</sup> jest trademarkiem zarejestrowanym przez Python Software Foundation.

Gurobi<sup>TM</sup> jest trademarkiem firmy Gurobi Optimization LLC zarejestrowanej w stanie Oregon, USA.

Styl kodu w dokumencie jest zaczerpnięty ze strony overleaf.

## 8 Radom incident

```
Removed virtual environment at: /env
Creating virtual environment at: /env
Installed 3 packages in 70ms
Traceback (most recent call last):
  File "/root/.app/optimization.py", line 32, in <module>
    np.random.seed(67)
  File "/root/.app/.venv/lib/python3.10/site-packages/numpy/_utils.py", line 792, in __getattr__
    raise AttributeError(f"module {__name__} has no attribute {attr!r}")
AttributeError: module 'numpy' has no attribute 'random'. Did you mean: 'random'?
```